

TAGD-03-sintesi

Filippo Visconti

Contents

1	SCALABILITÀ E ARCHITETTURE DISTRIBUITE	2
2	SAFETY vs LIVENESS (sistemi distribuiti non affidabili)	2
3	TEOREMA CAP (Eric Brewer)	2
4	PROBLEMA DEL CONSENSO	3
5	ALGORITMO RAFT	3
6	QUALITY OF SERVICE (QoS)	4
7	MODELLI DI SISTEMA (qualitativi)	5
8	RETI DI CODE (QN) — formalizzazione	7
9	ANALISI OPERAZIONALE — variabili e leggi	7
10	LEGGE DEL TEMPO DI RISPOSTA INTERATTIVO	8
11	MEAN VALUE ANALYSIS (MVA)	8
12	LIMITI DELLE PRESTAZIONI (Bottleneck Analysis)	8
13	CHECKLIST	9

1 SCALABILITÀ E ARCHITETTURE DISTRIBUITE

- **Scale up (verticale)**: potenziare un singolo server (CPU, RAM, disco). Limite: massimo fisico di risorse concentrabili in una macchina.
 - **Scale out (orizzontale)**: aumentare il numero di server, tramite **sharding** (partizionamento) e **replicazione**. Limite: communication overhead tra nodi.
 - **Partizionamento**: divisione logica/fisica dei dati tra nodi.
 - **Replicazione**: stessi dati su più nodi.
 - Mantenere consistenza tra repliche è la sfida principale.
-

2 SAFETY vs LIVENESS (sistemi distribuiti non affidabili)

- **Safety**: “non succede niente di male” → la **consistenza** è una proprietà di safety.
 - **Liveness**: “prima o poi succede qualcosa di buono” → la **disponibilità** è una proprietà di liveness.
 - **Unreliability**: fallimenti (network partition, crash, comportamenti Byzantine).
 - In un contesto non affidabile è **impossibile garantire simultaneamente** safety e liveness.
-

3 TEOREMA CAP (Eric Brewer)

Tre proprietà in tensione su un insieme di nodi distribuiti:

1. **Consistency** (atomic): le operazioni devono apparire come se eseguite su un singolo server centralizzato.
2. **Availability**: ogni richiesta riceve risposta (risposta troppo lenta = fault).
3. **Partition tolerance**: la rete può dividersi in gruppi non comunicanti tra loro (dato di fatto, non opzionale).

Enunciato: in presenza di partizionamento di rete, è impossibile avere una memoria condivisa read/write atomica che risponda sempre a ogni richiesta.

Dimostrazione intuitiva: se due nodi sono separati da partition, un update su uno non si propaga all'altro → l'altro nodo deve scegliere tra non rispondere (no availability) o rispondere con dato *stale* (no consistency).

3.1 Compromessi pratici

- **Best effort availability**: consistenza prioritaria, disponibilità sacrificata (indisponibile se partizionato).
- **Best effort consistency**: disponibilità/velocità prioritarie, si tollerano inconsistenze temporanee (eventual consistency).

3.2 Quorum-based consistency

- N_W = nodi minimi per confermare scrittura; N_R = nodi minimi per confermare lettura.
- $N_R + N_W > n$ → letture **strongly consistent** (intersezione read/write set sempre non vuota).
- $N_W > n/2$ → evita conflitti write-write (isolamento).

3.3 Trade-off dinamici (fattori che modulano consistenza/disponibilità)

- Soglie di “out-of-date-ness” (tollerare inconsistenza solo per partizioni brevi).
 - Stato dell'esecuzione (es. prenotazioni aeree: tollerante se molti posti, stretto se pochi).
 - Caratteristiche dati (inventario tollerante, carrello no).
 - Tipo di operazione (letture tolleranti, scritture no).
 - Utente/gerarchia geografica (consistenza maggiore entro partizione).
-

4 PROBLEMA DEL CONSENSO

Nodi $G_1, \dots, G_n$ con valore iniziale v_i devono accordarsi su un unico output.

Proprietà richieste: - **Agreement** (safety): tutti i nodi restituiscono lo stesso valore. - **Validity** (safety): il valore di output deve essere stato proposto da almeno un nodo. - **Termination** (liveness): ogni nodo deve restituire un valore prima o poi.

→ Safety garantita sempre; liveness garantita se la **maggioranza** dei nodi è attiva (anche con ritardi/partizioni/riordinamenti).

4.1 Replicated State Machine

- Ogni nodo ha copia identica di una macchina a stati, opera su un **log replicato**.
 - Macchine deterministiche + stessi comandi nello stesso ordine → stesso stato finale.
 - Il modulo di consenso mantiene allineati i log tra i nodi.
-

5 ALGORITMO RAFT

(Algoritmo di consenso pensato per essere più comprensibile di Paxos, adatto a sistemi reali — versione semplificata e didattica).

5.1 Tre sotto-problemi

1. **Leader election**
2. **Log replication**
3. **Safety** (nessun nodo applica comandi diversi per la stessa posizione)

5.2 Cluster e stati

- Tipicamente **5 server** → tollera 2 fallimenti simultanei.
- Stati: **Leader** (gestisce richieste client, follower redirige a lui), **Follower** (passivo), **Candidate** (temporaneo, durante elezioni).

5.3 Tempo e Term

- Tempo diviso in **term** (lunghezza arbitraria), ogni term inizia con un'elezione.
- Term = orologio logico per individuare informazioni obsolete.
- Server con term inferiore si aggiorna; leader/candidate che scopre term superiore → torna follower.
- Richieste con term obsoleto → rigettate.

5.4 Remote Procedure Calls

- **RequestVote**: usata dai candidate durante elezioni.
- **AppendEntries**: usata dai leader per replicare log e come heartbeat.

5.5 Invarianti di RAFT

- **Election safety**: max 1 leader per term.
- **Leader append-only**: leader non sovrascrive/cancella proprie entry.
- **Log matching**: stesso indice + stesso term → comando identico e log identici nelle posizioni precedenti.
- **Leader completeness**: entry committate in un term presenti nei leader di tutti i term successivi.
- **State machine safety**: nessuna macchina applica comandi diversi per lo stesso indice.

5.6 Processo di elezione

- Server parte come follower → se non riceve heartbeat entro election timeout diventa candidate, incrementa term, vota se stesso, invia RequestVote.
- Vince chi ottiene maggioranza.
- **Randomized election timeout** → evita split vote infiniti.

5.7 Replicazione del log

- Leader eletto riceve comandi dai client, li aggiunge al log, invia AppendEntries in parallelo.
- Entry **committed** quando replicata sulla maggioranza dei server.
- Commit di una entry → commit automatico di tutte le entry precedenti.
- Leader include indice committato più alto in ogni AppendEntries.

5.8 Gestione inconsistenze

- In caso di disallineamento (dovuto a crash), leader forza i follower a uniformarsi.
- Trova ultima entry coincidente (consistency check), cancella entry successive nel follower, invia le proprie.
- Leader mantiene **nextIndex** per ogni follower.

5.9 Restrizioni di safety

- Un votante **nega il voto** se il log del candidato è meno aggiornato del proprio (confronto: prima term, poi indice ultima entry).
- Leader **non committa mai** entry di term precedenti contando solo le repliche; commit solo per entry del term corrente (che porta con sé, per log matching, le entry precedenti).

5.10 Disponibilità/timing

- Safety indipendente dal tempo; disponibilità dipende da:

$$broadcastTime \ll electionTimeout \ll MTBF$$

- Se messaggi troppo lenti rispetto ai crash → impossibile eleggere leader stabile, niente progresso.

6 QUALITY OF SERVICE (QoS)

Requisiti **non funzionali** critici: prestazioni, disponibilità, sicurezza, manutenibilità. Fondamentali in: chiamate emergenza (112), trading online, applicazioni medicali.

6.1 Quattro attributi principali

Response time, Throughput, Availability, Scalability

6.2 Decomposizione del tempo di risposta (es. e-commerce)

1. **Browser Time** = Processing + I/O locale
2. **Network Time** = browser→ISP + Internet + ISP→server
3. **E-commerce Server Time** = Processing + I/O + networking lato server → congestione possibile in ogni fase.

6.3 Throughput

Tasso di richieste completate/unità di tempo. Unità per tipo sistema: - OLTP → tps - Web → req/s, page views/s, bytes/s - Router → pps, MB/s - CPU → MIPS, FLOPS - Storage → IOPS, KB/s

Calcolo throughput massimo: se I/O medio = 10ms (0.01s), a utilizzazione 100%:

$$Throughput_{max} = \frac{1}{0.01} = 100 \text{ IOPS}$$

Se workload = 60 IOPS → utilizzazione 60%. Approssimazione: $throughput = \min(capacity, workload)$
 Sopra la saturazione → **thrashing**: crollo prestazioni (overhead code/conflitti), invece di stabilizzarsi sul massimo.

6.4 Disponibilità

Frazione di tempo sistema attivo/accessibile.

Cause indisponibilità: guasti hardware, crash software (bug), sovraccarichi (richieste eccedenti → rifiuto, es. DBMS che rifiuta connessioni).

Admission Control: senza di esso il tempo di risposta cresce esponenzialmente col carico; con esso si accetta solo il carico gestibile, mantenendo risposte accettabili per le richieste accolte.

6.5 Scalabilità

Sistema scalabile = prestazioni non degradano significativamente con l'aumento di carico/utenti (crescita lineare/contenuta vs. impennata di un sistema non scalabile).

6.6 Ciclo di vita e QoS

1. **Requirements:** funzionali (cosa fa) vs non funzionali (quanto bene, es. “50 ricerche/sec, risposta <2s nel 95% dei casi”)
2. **System Design:** riuso componenti accelera funzionalità ma non garantisce performance
3. **Development:** testare prestazioni nel codice (es. join che falliscono su milioni di righe), instrumentare il codice
4. **Testing:** load test (modelli a ciclo aperto/chiuso) oltre a unit test
5. **Deployment:** fine-tuning configurazione
6. **Operation:** monitoraggio continuo

6.7 Categorie di metriche monitorate

- **Workload:** picchi, burstiness, anomalie (DoS), classi di richieste
- **Metriche esterne:** tempi risposta (media, dev. std, percentili), throughput, tasso scarto
- **Metriche interne:** utilizzazione CPU/mem/storage/rete, lunghezza code
- **Disponibilità:** tramite agenti esterni (richieste casuali, senza appesantire il sistema)

7 MODELLI DI SISTEMA (qualitativi)

- Sistemi = risorse condivise (CPU, disco, canali, thread, lock) → competizione → **code** → rete di code (**queuing networks**).
- Modello = astrazione; livello di dettaglio dipende dallo scopo. Unico modello perfetto = copia esatta del sistema (costoso/impraticabile).

7.1 Modelli Simulativi

- Programmi software che riproducono il comportamento dei componenti.
- Workload da traccia reale, benchmark sintetico, o distribuzioni di probabilità.
- $T = \frac{\sum T_i}{nt}$ (tempo risposta medio)
- Dettagliati e accurati ma costosi/complessi.

7.2 Modelli Analitici

- Formule matematiche/algoritmi.
- Meno dettagliati → meno accurati, ma efficienti e con parametri di input più semplici.

Approccio ibrido (capacity planning): simulativo per validare l'analitico, poi si usa l'analitico (errore tollerabile fino al ~30%).

7.3 Classi multiple di workload

Serve modello multi-classe quando:

1. Service demand eterogenei
2. Tipi diversi di workload (online vs batch)
3. SLO differenziati (es. 1.2s / 2.5s / 8s)

7.4 Classi Aperte

- Intensità = tasso di arrivi; popolazione potenzialmente illimitata, arrivi indipendenti dal numero già in sistema.
- Equilibrio: throughput = tasso di arrivo.

7.5 Classi Chiuse

- Intensità = popolazione fissa nel sistema (es. batch notturni, popolazione=5).
- Throughput è output, non input.

7.6 Modello Misto

Classi aperte + chiuse insieme (es. verificare SLO online mentre eseguono batch).

7.7 SLA (Service Level Agreement)

Esempio: disponibilità 99.99% (orario lavorativo) / 99.9% (resto); tempo risposta max 4s (non sicuro)/6s (sicuro); throughput min 2000 pagine/s; penali.

- **Risorsa Delay**: nessuna coda, servizio immediato (modella think time / risorse sovrabbondanti).
- **Risorsa Load-Dependent**: tasso di servizio dipende dal carico istantaneo (es. LAN condivisa).

7.8 Admission Control (dettaglio)

- W = limite massimo connessioni concorrenti (es. PostgreSQL `max_connections`, default 100).
- Se transazioni attuali = W → rifiuto.

$$Throughput = ArrivalRate \times (1 - ProbRejection)$$

7.9 Discipline di accodamento

- **FCFS**: ordine cronologico.
- **Priority Queuing**: priorità più alta servita prima (statica/dinamica, preemptive resume/restart).
- **Round Robin**: time slice fisso, poi passa al successivo.
- **Processor Sharing**: RR con quanti infinitesimali (parallelismo "illusorio").

7.10 Class Switching

Workload che cambia natura nel tempo (es. autenticazione: Disconnected → Connected/Authenticating → [denied→Disconnected | success→Connected/Authenticated] → fine transazione → Disconnected). Modellato con probabilità di transizione tra classi.

8 RETI DI CODE (QN) — formalizzazione

QN = collezione di K code interconnesse. Coda/stazione = waiting line + server.

8.1 Parametri di input

- **Intensità workload:** λ (classi aperte) o N (classi chiuse).
- **Service demand** $D_{i,r}$: tempo totale medio che la classe r passa nella risorsa i (di norma indipendente dal carico, con eccezioni).

8.2 Tipi di classi

- **Transaction:** aperta, intensità = esterno, utenti illimitati.
- **Batch:** chiusa, intensità = N fisso, un job entra appena uno termina.
- **Interactive:** chiusa, M terminali fissi + think time medio Z .

8.3 Notazione

- K = numero code; R = numero classi
- $\lambda = (\lambda_1, \dots, \lambda_R)$ (aperte); $N = (N_1, \dots, N_R)$ (chiuse)
- $D_{i,r}$ = service demand; $Prior(r)$ = priorità (1 = massima)
- Output: $X_{0,r}$ (throughput classe r), $R_{0,r}$ (tempo risposta classe r)
- Tipi coda: LI (load independent), LD (load dependent), Delay
- Discipline: FCFS, RR, PS

8.4 Aggregazione classi

λ_{new} = somma delle classi fuse. D_{new} = media pesata dei demand originali.

Esempio: Medio(=0.3) + Complesso(=0.2) $\rightarrow$ $\lambda_{new}=0.5$; $D_{CPU,new} = (0.3/0.5) \times 0.3 + (0.2/0.5) \times 0.45 = 0.36$

9 ANALISI OPERAZIONALE — variabili e leggi

9.1 Variabili fondamentali (periodo di osservazione T)

- T : durata osservazione
- K : numero risorse
- B_i : tempo occupazione risorsa i
- A_i : arrivi alla risorsa i ; A_0 : arrivi totali al sistema
- C_i : completamenti risorsa i ; C_0 : completamenti totali sistema

9.2 Metriche derivate

1. $S_i = \frac{B_i}{C_i}$ — tempo medio di servizio (esclude coda)
2. $U_i = \frac{B_i}{T}$ — utilizzazione
3. $X_i = \frac{C_i}{T}$ — throughput risorsa
4. $\lambda_i = \frac{A_i}{T}$ — tasso arrivi
5. $X_0 = \frac{C_0}{T}$ — throughput sistema

9.3 Leggi operazionali

Utilization Law: $U_i = S_i \times X_i$ (in equilibrio $A_i = C_i \Rightarrow \lambda_i = X_i$, quindi $U_i = S_i \times \lambda_i$) Con m server:
 $U_i = \frac{S_i \times X_i}{m}$

Service Demand Law: $D_i = \frac{B_i}{C_0} = \frac{U_i}{X_0} = S_i \times V_i$ (V_i = visite medie alla risorsa i)

Forced Flow Law: $X_i = X_0 \times V_i \rightarrow V_i = X_i/X_0$

Little's Law: relazione generale tra N (utenti nel sistema), R (tempo permanenza), X (tasso arrivo/completamento). Valida per ogni “scatola nera” senza creazione/distruzione di clienti.

Forme:

- Solo server: $N_{i,s} = U_i = X_i \times S_i$
- Centro completo: $N_i = X_i \times R_i$
- Sola coda: $N_{i,w} = X_i \times W_i$
- Sistema globale: $N_0 = X_0 \times R_0$

10 LEGGE DEL TEMPO DI RISPOSTA INTERATTIVO

Sistema con M client, ognuno alterna **think time (Z)** e **response time (R)**.

$\bar{M}$ = utenti in think phase; $\bar{N}$ = utenti in attesa risposta.

$M = \bar{M} + \bar{N}$; per Little: $\bar{M} = X_0 \times Z$, $\bar{N} = X_0 \times R$

$$M = X_0(Z + R) \Rightarrow R = \frac{M}{X_0} - Z$$

10.1 Riepilogo leggi

Legge	Formula
Utilization	$U_i = X_i \times S_i = \lambda_i \times S_i$
Forced Flow	$X_i = V_i \times X_0$
Service Demand	$D_i = V_i \times S_i = U_i / X_0$
Little	$N = X \times R$
Interactive Response Time	$R = M / X_0 - Z$

11 MEAN VALUE ANALYSIS (MVA)

Algoritmo **ricorsivo** per risolvere reti di code **chiuse**: calcola tempo di residenza, tempo di risposta, throughput sistema/centri, utilizzazione, numero medio job.

11.1 Passi algoritmo

1. Init: $n_i(0) = 0$ per ogni centro
2. Per $n=1 \dots N$:
 - a. $R'_i(n) = D_i[1 + n_i(n-1)]$ (tempo di residenza)
 - b. $R_0(n) = \sum_i R'_i(n)$
 - c. $X_0(n) = n / R_0(n)$ (Little)
 - d. $X_i(n) = V_i \times X_0(n)$ (Forced Flow)
 - e. $U_i(n) = S_i \times X_i(n)$ (Utilization)
 - f. $n_i(n) = X_i(n) \times R'_i(n)$ (Little)

12 LIMITI DELLE PRESTAZIONI (Bottleneck Analysis)

12.1 Identificazione bottleneck

Risorsa con **utilizzazione più alta** = risorsa con **service demand più alto** ($\max D_i$).

12.2 Throughput massimo

Da $U_i = D_i \times X_0$: $X_0 = U_i/D_i$. Max quando $U_i = 1$ (100%):

$$X_{0,max} = \frac{1}{\max D_i}$$

Upper bound asintotico completo - **Heavy load** (saturazione): $X_0 \leq \frac{1}{\max D_i}$ - **Light load** (nessuna attesa in coda): $R = \sum_i D_i$; con Little ($N = R \times X_0$): $X_0 \leq \frac{N}{\sum_i D_i}$ - **Bound combinato**:

$$X_0 \leq \min \left[\frac{N}{\sum_i D_i}, \frac{1}{\max D_i} \right]$$

12.3 Lower bound sul tempo di risposta

Da Little, sostituendo il bound su X_0 :

$$R \geq \max \left[\sum_i D_i, N \times \max D_i \right]$$

Graficamente: curva orizzontale ($\sum D_i$) per N piccoli, poi retta crescente con pendenza $\max D_i$ per N grandi. Il tempo di risposta reale è sempre **superiore** a questo limite.

13 CHECKLIST

- ☐ Scale up vs scale out, partizionamento vs replicazione
- ☐ Safety vs Liveness, Unreliability
- ☐ Teorema CAP: enunciato, dimostrazione intuitiva, esempi (Chubby, Akamai)
- ☐ Quorum: $N_R + N_W > n$, $N_W > n/2$
- ☐ Consenso: Agreement/Validity/Termination, Replicated State Machine
- ☐ RAFT: stati, term, RPC, invarianti, elezione, replicazione log, safety restriction, broadcast-Time election Timeout MTBF
- ☐ QoS: response time, throughput, availability, scalability; thrashing; admission control
- ☐ Ciclo di vita QoS (6 fasi)
- ☐ Modelli simulativi vs analitici
- ☐ Classi aperte vs chiuse vs interactive vs mixed; class switching
- ☐ QN: notazione K, R, N, $D_{\{i,r\}}$, V_i
- ☐ Variabili operazionali: T, B, A, C
- ☐ 5 leggi operazionali (Utilization, Forced Flow, Service Demand, Little, Interactive Response Time)
 - saperle applicare a memoria
- ☐ Algoritmo MVA passo-passo
- ☐ Bottleneck analysis: upper bound throughput, lower bound response time, ribilanciamento vs upgrade hardware